# Homework 02

## Due Date: Feb 20, 2026, 11:00PM

## Student's Name: Dayne Lee

## Student's e-mail: dl5635@nyu.edu

## Problem 1. (15 points)

- 1.1. (5 points)
  By using **count from itertools** make a generator which produces all numbers from 0 to 1000 (including the two) divisible by 5.

- 1.2. (5 points)
  From the obtained generator produce a list of numbers that are not divisible by 25.

- 1.3. (5 points)
  Print the list and the length of the list.

In [1]:
```
from itertools import count, islice

# 1.1
# [n for n in islice(count(0), 1001) if n % 5 == 0]
div_by_5 = (n for n in islice(count(0), 1001) if n % 5 == 0)

# 1.2
not_div_by_25 = [n for n in div_by_5 if n % 25 != 0]

# 1.3
print('list: ', not_div_by_25)
print('length of the list: ', len(not_div_by_25))
```

```
list:  [5, 10, 15, 20, 30, 35, 40, 45, 55, 60, 65, 70, 80, 85, 90, 95, 105, 110, 115, 120, 130, 135, 140, 1
45, 155, 160, 165, 170, 180, 185, 190, 195, 205, 210, 215, 220, 230, 235, 240, 245, 255, 260, 265, 270, 28
0, 285, 290, 295, 305, 310, 315, 320, 330, 335, 340, 345, 355, 360, 365, 370, 380, 385, 390, 395, 405, 410,
415, 420, 430, 435, 440, 445, 455, 460, 465, 470, 480, 485, 490, 495, 505, 510, 515, 520, 530, 535, 540, 54
5, 555, 560, 565, 570, 580, 585, 590, 595, 605, 610, 615, 620, 630, 635, 640, 645, 655, 660, 665, 670, 680,
685, 690, 695, 705, 710, 715, 720, 730, 735, 740, 745, 755, 760, 765, 770, 780, 785, 790, 795, 805, 810, 81
5, 820, 830, 835, 840, 845, 855, 860, 865, 870, 880, 885, 890, 895, 905, 910, 915, 920, 930, 935, 940, 945,
955, 960, 965, 970, 980, 985, 990, 995]
length of the list:  160
```

## Problem 2. (20 points)

Write three functions to calculate n! (n factorial) for a given non-negative integer n:

- 2.1 (5 points)
  f_rec(n) in a recursive way (in a simplest way without memoization)
- 2.2 (5 points)
- f_it(n) in an iterative way
- 2.3 (10 points)
- f_acc(n) by using "accumulate" from itertools.

Note the ratios of the expected run-times:

1. f_rec(n=1000) and f_it(n=1000): 356 / 271 = 1.31
2. f_acc(n=1000) and f_it(n=1000): 283 / 271 = 1.04

In [2]:
```
# 2.1
def f_rec(n):
    if n == 0 or n == 1:
        return 1
    return n * f_rec(n-1)

# 2.2
def f_it(n):
    res = 1
```

```python
    for i in range(2, n+1):
        res *= i

    return res

# 2.3
from itertools import accumulate
import operator


def f_acc(n):
    res = 1
    for j in accumulate(range(2, n+1), operator.mul):
        res = j

    return res
```

In [3]:
```python
%timeit -r 7 -n 1000 f_it(1000)
%timeit -r 7 -n 1000 f_rec(1000)
%timeit -r 7 -n 1000 f_acc(1000)
```

```
271 µs ± 7.89 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
356 µs ± 1.78 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
283 µs ± 12.5 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

# Problem 3 (35 points)

- 3.1 (5 points)
  Write a function `perm_generator(n)`, which for a given positive integer $n$, returns a generator with all permutations of the set $\{0, 1, 2, \ldots n-1\}$.

- 3.2 (10 points)
  Write a function `comb_generator(n, k)`, which for given non-negative integers $n, k$ where $n \geq k$, returns a generator with all combinations of size $k$ on the set $\{0, 1, 2, \ldots n-1\}$.

- 3.3 (20 points)
  Write a function `h_generator(n)`, which for a given positive integer $n$, returns a generator with all permutations of the set $\{0, 1, 2, \ldots n-1\}$ such that for all $i = 0, 1, 2, \ldots, n-1$ the number $i$ is not at the $i$-th position.

- 3.3. Example: For $n = 3$, there are $3! = 6$ permutations $(0, 1, 2), (0, 2, 1), (1, 0, 2), (1, 2, 0), (2, 0, 1), (2, 1, 0)$. And `h_generator(n)` should return the generator with permutations $(1, 2, 0)$ and $(2, 0, 1)$. For example, $(2, 1, 0)$ cannot be included since here $1$ is at the $1$-st position.

In [4]:
```
from itertools import permutations, combinations

# 3.1
def perm_generator(n):
    assert n > 0
    return permutations(range(n))

# 3.2
def comb_generator(n, k):
    assert n >= 0 and k >= 0 and n >= k
    return combinations(range(n), k)

# 3.3
def h_generator(n):
    assert n > 0

    for perm in permutations(range(n)):
        if all(i != k for i, k in enumerate(perm)):
            yield perm
```

In [5]:
```
print("type(h_generator(3)): ", type(h_generator(3)))
list(h_generator(3))
```

```
type(h_generator(3)):  <class 'generator'>
```

Out[5]:
```
[(1, 2, 0), (2, 0, 1)]
```

## Problem 4 (30 points)

By using **starmap** from **itertools** write a function that returns a generator that produces all distances for a given input list of points, in the two-dimensional coordinate system.

In more detail, for a given input list of points $\text{list_of_points} = [p_0, p_1, \ldots, p_{n-1}]$, write a function:

$$\text{f_distances(list_of_points)}$$

that returns a generator that produces tuples $(p_i, p_j, ||p_i - p_j||)$, (where each tuple stores the point i, point j and their Euclidean distance). (There are $n(n-1)/2$ tuples.)

For example:

- input: For 3 points as $\text{list_of_points} = [(0,0),(0,1),(1,0)]$
- output: A generator that produces $\binom{3}{2} = 3$ tuples: $((0,0),(0,1),1); ((0,0),(1,0),1); ((0,1),(1,0),\sqrt{2})$.

In [6]:
```
from itertools import starmap, combinations

def f_distance(points):
    combs = combinations(points, 2)
    dist = lambda p1, p2: ((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)**(1/2)
    return starmap(lambda p1, p2: (p1, p2, dist(p1, p2)), combs)
```

In [7]:
```
points = [(0, 0), (0, 1), (1, 0)]

print("type(f_distance(points)): ", type(f_distance(points)))
list(f_distance(points))
```

```
type(f_distance(points)):  <class 'itertools.starmap'>
```

Out[7]:
```
[((0, 0), (0, 1), 1.0),
 ((0, 0), (1, 0), 1.0),
 ((0, 1), (1, 0), 1.4142135623730951)]
```